Tareas de la unidad 1

Desarrollo del software · Entornos de Desarrollo · 1.º DAW

Índice

1 Cómo funcionan estas tareas **1**

2 T1.1 · ¿Qué tipo de programa es cada uno? **3**
2.1 Qué tienes que rellenar . . . 3
2.2 Y además . . . 3
2.3 Qué se entrega . . . 3

3 T1.2 · ¿Quién traduce cada lenguaje? **4**
3.1 Qué tienes que rellenar . . . 4
3.2 Y además . . . 4
3.3 Qué se entrega . . . 4

4 T1.3 · Fibonacci en tres lenguajes **5**
4.1 Qué tienes que entregar de cada uno . . . 5
4.2 Y después, responde a esto . . . 5
4.3 Qué se entrega . . . 5

5 T1.4 · Preguntas cortas de teoría **6**
5.1 Software y tipos . . . 6
5.2 Código fuente y programa . . . 6
5.3 Los traductores . . . 6
5.4 Lenguajes . . . 6
5.5 El proceso . . . 6

1 Cómo funcionan estas tareas

Cuatro tareas cortas de la UD1. Las tres primeras son de buscar información; la cuarta es de repasar la teoría.

Tarea	De qué va	Tiempo	Entrega
T1.1	Clasificar nueve programas por su tipo	~40 min	Moodle
T1.2	Quién traduce cada lenguaje	~30 min	Moodle
T1.3	La sucesión de Fibonacci en tres lenguajes	~30 min	Moodle
T1.4	Preguntas cortas de teoría	~30 min	Moodle

Ojo

Tres reglas para las tres primeras, que son de búsqueda:

1. **Pon siempre la dirección web de donde has sacado cada dato.** Un dato sin fuente no cuenta.

2. **No copies y pegues sin entender.** En clase te voy a preguntar por lo que has entregado, y si no sabes explicar lo que has escrito, la tarea no cuenta aunque esté bien.
3. **Puedes usar inteligencia artificial**, pero tienes que decir dónde la has usado y comprobar lo que te diga. Se equivoca bastante con los nombres de herramientas y con las versiones.

2 T1.1 · ¿Qué tipo de programa es cada uno?

Unos 40 minutos. Individual.

Busca **nueve programas reales** que **no hayan salido en clase**: tres de cada tipo.

Tipo	Cuántos
Software de sistema	3
Software de programación	3
Software de aplicación	3

2.1 Qué tienes que rellenar

Para cada uno de los nueve:

Columna	Qué poner
Nombre	El nombre oficial del programa
Quién lo hace	Empresa, fundación o comunidad que lo desarrolla
Web oficial	La dirección. La oficial, no la de un blog
Para qué sirve	**Una sola frase**, con tus palabras
Tipo	Sistema · Programación · Aplicación
Libre o propietario	Y di qué licencia tiene, si la encuentras

2.2 Y además

1. **Elige los DOS que más te hayan costado clasificar** y explica en cinco líneas cada uno por qué dudabas y por qué te has decidido al final. *Esta parte vale tanto como la tabla entera: los casos claros no demuestran nada.*
2. Busca **un programa que se venda como estándar pero que se pueda personalizar mucho** —tipo un ERP o un gestor de contenidos—. Di cuál es y explica en tres líneas por qué no es software a medida aunque lo parezca.

Idea clave

Para que no te quedes en blanco, estos sitios valen: la web oficial del fabricante, la Wikipedia en inglés (suele ser más precisa para software), `alternativeto.net` para encontrar programas parecidos a uno que ya conoces, y `sourceforge.net` o `github.com` si buscas software libre.
Lo que NO vale: una lista de «los 10 mejores programas de…» de una web de anuncios, sin entrar en la web oficial a comprobar el dato.

2.3 Qué se entrega

Un documento con la tabla de nueve filas, los dos párrafos de los casos dudosos y el párrafo del software personalizable. **Máximo dos páginas.**

3 T1.2 · ¿Quién traduce cada lenguaje?

Unos 30 minutos. Se puede hacer por parejas.

Para cada uno de estos **siete lenguajes**, averigua quién hace la traducción y qué sale de ella.

`C` · `C++` · `Java` · `Python` · `C#` · `JavaScript` · `Go`

3.1 Qué tienes que rellenar

Columna	Qué poner
Lenguaje	
Traductor más usado	El nombre del programa que hace la traducción
¿Compilador, intérprete o mixto?	
Qué genera	Un ejecutable · un código intermedio · nada
Quién lo mantiene	Empresa, fundación o comunidad
¿Es gratuito?	
Otro traductor alternativo	Para casi todos hay más de uno

3.2 Y además

1. **Un lenguaje con dos traductores muy distintos.** Elige uno de los siete que tenga al menos dos traductores conocidos, nómbralos y explica en cinco líneas en qué se diferencian y por qué existen los dos.
2. **La pregunta del cliente.** Trabajas en una empresa y tenéis que entregarle un programa a un cliente que **no debe poder ver vuestro código fuente**. De los siete lenguajes, ¿cuáles te valdrían sin más y cuáles te darían problema? Justifícalo en cinco líneas.

Ojo

Cuidado con dos trampas de esta tarea:

- **JavaScript.** Hoy no hay un solo traductor y la respuesta «es interpretado» se queda corta. Investiga qué hace realmente el motor del navegador. Si llegas a la palabra **JIT**, vas bien.
- **Java y C#** no encajan en la casilla de compilado ni en la de interpretado. Esa es precisamente la gracia: fíjate en qué fichero generan.

3.3 Qué se entrega

La tabla de siete filas y los dos párrafos. **Una página y media como mucho.**

4 T1.3 · Fibonacci en tres lenguajes

Unos 30 minutos. Individual.

La **sucesión de Fibonacci** es esta: cada número es la suma de los dos anteriores.

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...
```

Busca en internet un programa que calcule esa sucesión, **en tres lenguajes distintos**, y que cumplan esto:

Debe haber	Por ejemplo
Uno de un lenguaje **compilado**	C, C++, Go, Rust
Uno de un lenguaje **interpretado**	Python, PHP, Ruby
Uno de un lenguaje **mixto**	Java, C#, JavaScript

4.1 Qué tienes que entregar de cada uno

1. El **código**, copiado tal cual. Dilo si lo has tenido que adaptar.
2. La **dirección web exacta** de donde lo has sacado.
3. Si esa página dice algo sobre la **licencia** del código, cópialo. Si no dice nada, escríbelo también: «la página no indica licencia».

4.2 Y después, responde a esto

1. **¿Qué se parece en los tres?** Encuentra al menos dos cosas que aparezcan en los tres programas aunque estén escritas de forma distinta.
2. **¿Qué cambia?** Al menos dos diferencias claras entre unos y otros.
3. **Cuenta las líneas de cada uno.** ¿El más corto es el más fácil de entender? Responde con tu opinión y explícala.
4. **Lo importante:** para cada uno de los tres, di **qué hace falta tener instalado** en un ordenador para poder ejecutarlo, y **qué fichero se genera**, si es que se genera alguno.
5. Si en alguno de los tres has encontrado la palabra **recursivo** o **recursiva**, di qué crees que significa. Si no la has encontrado en ninguno, busca una versión recursiva de uno de ellos y añádela.

Idea clave

Esto no es una tarea de programación. No hace falta que entiendas el código línea a línea ni que sepas ejecutarlo. Lo que se evalúa es que sepas mirar tres programas que hacen exactamente lo mismo y ver qué parte es «el problema» y qué parte es «el lenguaje».

Ojo

Cita las fuentes de verdad. Copiar código de internet sin decir de dónde viene no es un detalle formal: dependiendo de la licencia, puede ser una infracción. Lo veremos en serio más adelante, pero se empieza a hacer bien desde hoy.

4.3 Qué se entrega

Un documento con los tres códigos, sus tres direcciones web, las cinco respuestas y, al final, una frase diciendo si has usado inteligencia artificial y para qué.

5 T1.4 · Preguntas cortas de teoría

Unos 30 minutos. Individual y sin apuntes la primera vuelta.

Contesta con **una o dos frases** cada una. No hace falta más: se valora la precisión, no la extensión.

5.1 Software y tipos

1. Da **dos** de los cuatro rasgos que definen al software frente a cualquier otro producto.
2. Explica en una frase por qué se dice que **el software no se estropea**.
3. ¿Qué diferencia hay entre un **programa** y una **aplicación**?
4. Di un ejemplo de software de **sistema**, uno de **programación** y uno de **aplicación**, y que ninguno de los tres sea de los que salieron en clase.
5. Una empresa compra un programa de gestión que usan miles de empresas más, y luego lo configura para su forma de trabajar. ¿Eso es software **a medida** o **estándar**? Justifícalo.

5.2 Código fuente y programa

6. ¿Por qué se dice que un programa **no se escribe, se obtiene**?
7. ¿Cuál de estos dos se puede abrir con el Bloc de notas y leer: el **código fuente** o el **ejecutable**? ¿Por qué?
8. ¿Qué le falta al **código objeto** para poder ejecutarse?
9. ¿Cómo se llama el programa que le añade eso que le falta?
10. Ordena: *ejecutable* · *código fuente* · *código objeto*.

5.3 Los traductores

11. Explica la diferencia entre **compilador** e **intérprete** en dos frases.
12. Con un compilador, ¿cuándo te enteras de que tienes un error? ¿Y con un intérprete?
13. Si le entregas a un cliente un programa hecho en un lenguaje **interpretado**, ¿le estás dando también tu código fuente? ¿Por qué?
14. ¿Qué genera un traductor **mixto** en lugar de un ejecutable?
15. ¿Para qué sirve la **máquina virtual** en la vía mixta?
16. Java se creó buscando «escribir una vez, ejecutar en cualquier sitio». Explica en dos frases **cómo lo consigue**.

5.4 Lenguajes

17. Ordena de más cerca de la máquina a más cerca de la persona: *alto nivel* · *lenguaje máquina* · *ensamblador*.
18. Di los **cuatro criterios** con los que se puede clasificar un lenguaje.
19. Clasifica **Java** según esos cuatro criterios.
20. ¿Qué significa que un lenguaje sea **multiparadigma**? Pon un ejemplo.

5.5 El proceso

21. Nombra **cuatro** de las siete fases del ciclo de vida del software.
22. ¿En qué fase se va la mayor parte del dinero de un programa a lo largo de su vida? ¿Sabrías decir aproximadamente cuánto?
23. ¿Cuál es el problema del modelo en **cascada**?
24. Cita **dos** herramientas del proceso de desarrollo y di qué hace cada una.